

Chapter

1

Introduction to Databases

Chapter Objectives

In this chapter you will learn:

- Some common uses of database systems.
- The characteristics of file-based systems.
- The problems with the file-based approach.
- The meaning of the term 'database'.
- The meaning of the term 'database management system' (DBMS).
- The typical functions of a DBMS.
- The major components of the DBMS environment.
- The personnel involved in the DBMS environment.
- The history of the development of DBMSs.
- The advantages and disadvantages of DBMSs.

The history of database system research is one of exceptional productivity and startling economic impact. Barely 20 years old as a basic science research field, database research has fueled an information services industry estimated at \$10 billion per year in the U.S. alone. Achievements in database research underpin fundamental advances in communications systems, transportation and logistics, financial management, knowledge-based systems, accessibility to scientific literature, and a host of other civilian and defense applications. They also serve as the foundation for considerable progress in the basic science fields ranging from computing to biology.

(Silberschatz *et al.*, 1990, 1996)

This quotation is from a workshop on database systems at the beginning of the 1990s and expanded upon in a subsequent workshop in 1996, and it provides substantial motivation for the study of the subject of this book: the **database system**. Since these workshops, the importance of the database system has, if anything, increased with the significant developments in hardware capability, hardware capacity, and communications, including the

emergence of the Internet, electronic commerce, business intelligence, mobile communications, and grid computing. The database system is arguably the most important development in the field of software engineering, and the database is now the underlying framework of the information system, fundamentally changing the way that many organizations operate. Database technology has been an exciting area to work in and, since its emergence, has been the catalyst for many important developments in software engineering. The workshop emphasized that the developments in database systems were not over, as some people thought. In fact, to paraphrase an old saying, it may be that we are only *at the end of the beginning* of the development. The applications that will have to be handled in the future are so much more complex that we will have to rethink many of the algorithms currently being used, such as the algorithms for file storage and access, and query optimization. The development of these original algorithms has had significant ramifications in software engineering and, without doubt, the development of new algorithms will have similar effects. In this first chapter we introduce the database system.

Structure of this Chapter

In Section 1.1 we examine some uses of database systems that we find in everyday life but are not necessarily aware of. In Sections 1.2 and 1.3 we compare the early file-based approach to computerizing the manual file system with the modern, and more usable, database approach. In Section 1.4 we discuss the four types of role that people perform in the database environment, namely: data and database administrators, database designers, application developers, and the end-users. In Section 1.5 we provide a brief history of database systems, and follow that in Section 1.6 with a discussion of the advantages and disadvantages of database systems.

Throughout this book, we illustrate concepts using a case study based on a fictitious property management company called *DreamHome*. We provide a detailed description of this case study in Section 10.4 and Appendix A. In Appendix B we present further case studies that are intended to provide additional realistic projects for the reader. There will be exercises based on these case studies at the end of many chapters.

1.1 Introduction

The database is now such an integral part of our day-to-day life that often we are not aware we are using one. To start our discussion of databases, in this section we examine some applications of database systems. For the purposes of this discussion, we consider a *database* to be a collection of related data and the *Database Management System* (DBMS) to be the software that manages and controls access to the database. A *database application* is simply a program that interacts with the database at some point in its execution. We also use the more inclusive term *database system* to be a collection of application programs that interact with the database along with the DBMS and database itself. We provide more accurate definitions in Section 1.3.

Purchases from the supermarket

When you purchase goods from your local supermarket, it is likely that a database is accessed. The checkout assistant uses a bar code reader to scan each of your purchases. This is linked to an application program that uses the bar code to find out the price of the item from a product database. The program then reduces the number of such items in stock and displays the price on the cash register. If the reorder level falls below a specified threshold, the database system may automatically place an order to obtain more stocks of that item. If a customer telephones the supermarket, an assistant can check whether an item is in stock by running an application program that determines availability from the database.

Purchases using your credit card

When you purchase goods using your credit card, the assistant normally checks that you have sufficient credit left to make the purchase. This check may be carried out by telephone or it may be carried out automatically by a card reader linked to a computer system. In either case, there is a database somewhere that contains information about the purchases that you have made using your credit card. To check your credit, there is a database application program that uses your credit card number to check that the price of the goods you wish to buy together with the sum of the purchases you have already made this month is within your credit limit. When the purchase is confirmed, the details of the purchase are added to this database. The application program also accesses the database to check that the credit card is not on the list of stolen or lost cards before authorizing the purchase. There are other application programs to send out monthly statements to each cardholder and to credit accounts when payment is received.

Booking a holiday at the travel agents

When you make inquiries about a holiday, the travel agent may access several databases containing holiday and flight details. When you book your holiday, the database system has to make all the necessary booking arrangements. In this case, the system has to ensure that two different agents do not book the same holiday or overbook the seats on the flight. For example, if there is only one seat left on the flight from London to New York and two agents try to reserve the last seat at the same time, the system has to recognize this situation, allow one booking to proceed, and inform the other agent that there are now no seats available. The travel agent may have another, usually separate, database for invoicing.

Using the local library

Your local library probably has a database containing details of the books in the library, details of the readers, reservations, and so on. There will be a computerized index that allows readers to find a book based on its title, or its authors, or its subject area. The database system handles reservations to allow a reader to reserve a book and to be informed by mail when the book is available. The system also sends reminders to borrowers who have failed to return books by the due date. Typically, the system will have a bar code

reader, similar to that used by the supermarket described earlier, which is used to keep track of books coming in and going out of the library.

Taking out insurance

Whenever you wish to take out insurance, for example personal insurance, building, and contents insurance for your house, or car insurance, your broker may access several databases containing figures for various insurance organizations. The personal details that you supply, such as name, address, age, and whether you drink or smoke, are used by the database system to determine the cost of the insurance. The broker can search several databases to find the organization that gives you the best deal.

Renting a video

When you wish to rent a video from a video rental company, you will probably find that the company maintains a database consisting of the video titles that it stocks, details on the copies it has for each title, whether the copy is available for rent or whether it is currently on loan, details of its members (the renters), and which videos they are currently renting and date they are returned. The database may even store more detailed information on each video, such as its director and its actors. The company can use this information to monitor stock usage and predict future buying trends based on historic rental data.

Using the Internet

Many of the sites on the Internet are driven by database applications. For example, you may visit an online bookstore that allows you to browse and buy books, such as Amazon.com. The bookstore allows you to browse books in different categories, such as computing or management, or it may allow you to browse books by author name. In either case, there is a database on the organization's Web server that consists of book details, availability, shipping information, stock levels, and on-order information. Book details include book titles, ISBNs, authors, prices, sales histories, publishers, reviews, and detailed descriptions. The database allows books to be cross-referenced: for example, a book may be listed under several categories, such as computing, programming languages, bestsellers, and recommended titles. The cross-referencing also allows Amazon to give you information on other books that are typically ordered along with the title you are interested in.

As with an earlier example, you can provide your credit card details to purchase one or more books online. Amazon.com personalizes its service for customers who return to its site by keeping a record of all previous transactions, including items purchased, shipping, and credit card details. When you return to the site, you can now be greeted by name and you can be presented with a list of recommended titles based on previous purchases.

Studying at university

If you are at university, there will be a database system containing information about yourself, the course you are enrolled in, details about your grant, the modules you have taken in previous years or are taking this year, and details of all your examination results. There

may also be a database containing details relating to the next year's admissions and a database containing details of the staff who work at the university, giving personal details and salary-related details for the payroll office.

Traditional File-Based Systems

1.2

It is almost a tradition that comprehensive database books introduce the database system with a review of its predecessor, the file-based system. We will not depart from this tradition. Although the file-based approach is largely obsolete, there are good reasons for studying it:

- Understanding the problems inherent in file-based systems may prevent us from repeating these problems in database systems. In other words, we should learn from our earlier mistakes. Actually, using the word 'mistakes' is derogatory and does not give any cognizance to the work that served a useful purpose for many years. However, we have learned from this work that there are better ways to handle data.
- If you wish to convert a file-based system to a database system, understanding how the file system works will be extremely useful, if not essential.

File-Based Approach

1.2.1

File-based system	A collection of application programs that perform services for the end-users such as the production of reports. Each program defines and manages its own data.
--------------------------	--

File-based systems were an early attempt to computerize the manual filing system that we are all familiar with. For example, in an organization a manual file is set up to hold all external and internal correspondence relating to a project, product, task, client, or employee. Typically, there are many such files, and for safety they are labeled and stored in one or more cabinets. For security, the cabinets may have locks or may be located in secure areas of the building. In our own home, we probably have some sort of filing system which contains receipts, guarantees, invoices, bank statements, and such like. When we need to look something up, we go to the filing system and search through the system starting from the first entry until we find what we want. Alternatively, we may have an indexing system that helps locate what we want more quickly. For example, we may have divisions in the filing system or separate folders for different types of item that are in some way *logically related*.

The manual filing system works well while the number of items to be stored is small. It even works quite adequately when there are large numbers of items and we have only to store and retrieve them. However, the manual filing system breaks down when we have to cross-reference or process the information in the files. For example, a typical real estate agent's office might have a separate file for each property for sale or rent, each potential buyer and renter, and each member of staff. Consider the effort that would be required to answer the following questions:

- What three-bedroom properties do you have for sale with a garden and garage?
- What flats do you have for rent within three miles of the city center?
- What is the average rent for a two-bedroom flat?
- What is the total annual salary bill for staff?
- How does last month's turnover compare with the projected figure for this month?
- What is the expected monthly turnover for the next financial year?

Increasingly, nowadays, clients, senior managers, and staff want more and more information. In some areas there is a legal requirement to produce detailed monthly, quarterly, and annual reports. Clearly, the manual system is inadequate for this type of work. The file-based system was developed in response to the needs of industry for more efficient data access. However, rather than establish a centralized store for the organization's operational data, a decentralized approach was taken, where each department, with the assistance of **Data Processing (DP)** staff, stored and controlled its own data. To understand what this means, consider the *DreamHome* example.

The Sales Department is responsible for the selling and renting of properties. For example, whenever a client approaches the Sales Department with a view to marketing his or her property for rent, a form is completed, similar to that shown in Figure 1.1(a). This gives details of the property such as address and number of rooms together with the owner's details. The Sales Department also handles inquiries from clients, and a form similar to the one shown in Figure 1.1(b) is completed for each one. With the assistance of the DP Department, the Sales Department creates an information system to handle the renting of property. The system consists of three files containing property, owner, and client details, as illustrated in Figure 1.2. For simplicity, we omit details relating to members of staff, branch offices, and business owners.

The Contracts Department is responsible for handling the lease agreements associated with properties for rent. Whenever a client agrees to rent a property, a form is filled in by one of the Sales staff giving the client and property details, as shown in Figure 1.3. This form is passed to the Contracts Department which allocates a lease number and completes the payment and rental period details. Again, with the assistance of the DP Department, the Contracts Department creates an information system to handle lease agreements. The system consists of three files storing lease, property, and client details, containing similar data to that held by the Sales Department, as illustrated in Figure 1.4.

The situation is illustrated in Figure 1.5. It shows each department accessing their own files through application programs written specially for them. Each set of departmental application programs handles data entry, file maintenance, and the generation of a fixed set of specific reports. What is more important, the physical structure and storage of the data files and records are defined in the application code.

We can find similar examples in other departments. For example, the Payroll Department stores details relating to each member of staff's salary, namely:

```
StaffSalary(staffNo, fName, lName, sex, salary, branchNo)
```

The Personnel Department also stores staff details, namely:

```
Staff(staffNo, fName, lName, position, sex, dateOfBirth, salary, branchNo)
```

DreamHome Client Details Client Number: CR74	
First Name Mike	Last Name Ritchie
Address 18 Tain St, PA1G 1YQ	Tel No. 01475-392178
Property Requirement Details	
Preferred Property Type House	Maximum Monthly Rent 750
General Comments Currently living at home with parents Getting married in August	
Seen By Ann Beech	Date 24-Mar-04
Branch No B003	Branch City Glasgow

(b)

DreamHome Property for Rent Details Property Number: PG21	
Address 18 Dale Rd	Allocated to Branch: 163 Main St, Glasgow
City Glasgow	Branch No B003
Postcode G12	Staff Responsible Ann Beech
Type House	Rent 600
No of Rooms 5	
Owner's Details	
Name Carol Farrel	Business Name
Address 6 Achray St, Glasgow G32 9DX	Address
Tel No. 0141-357-7419	Tel No.
Owner No. C087	Owner No.
	Contact Name
	Business Type

(a)

Figure 1.1 Sales Department forms: (a) Property for Rent Details form; (b) Client Details form.

Figure 1.2

The PropertyForRent, PrivateOwner, and Client files used by Sales.

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93

PrivateOwner

ownerNo	fName	lName	address	telNo
CO46	Joe	Keogh	2 Fergus Dr, Aberdeen AB2 7SX	01224-861212
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

Client

clientNo	fName	lName	address	telNo	prefType	maxRent
CR76	John	Kay	56 High St, London SW1 4EH	0207-774-5632	Flat	425
CR56	Aline	Stewart	64 Fern Dr, Glasgow G42 0BL	0141-848-1825	Flat	350
CR74	Mike	Ritchie	18 Tain St, PA1G 1YQ	01475-392178	House	750
CR62	Mary	Tregear	5 Tarbot Rd, Aberdeen AB9 3ST	01224-196720	Flat	600

Figure 1.3

Lease Details form used by Contracts Department.

DreamHome
Lease Details
Lease Number: 10012

Client No. CR74

Full Name Mike Ritchie

Address (previous) 18 Tain St,
PA1G 1YQ

Tel No. 01475-392178

Property No. PG21

Address 18 Dale Rd,
Glasgow G12

Payment Details

Monthly Rent 600

Payment Method Cheque

Deposit 1200 Paid (Y or N) Y

Rent Start Date 1-Jul-04

Rent Finish Date 30-Jun-05

Duration 1Year

Lease

leaseNo	propertyNo	clientNo	rent	payment Method	deposit	paid	rentStart	rentFinish	duration
10024	PA14	CR62	650	Visa	1300	Y	1-Jun-05	31-May-05	12
10075	PL94	CR76	400	Cash	800	N	1-Aug-05	31-Jan-05	6
10012	PG21	CR74	600	Cheque	1200	Y	1-Jul-05	30-Jun-05	12

PropertyForRent

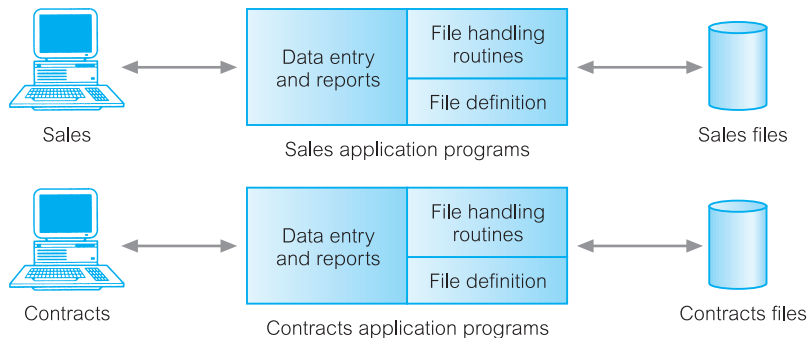
propertyNo	street	city	postcode	rent
PA14	16 Holhead	Aberdeen	AB7 5SU	650
PL94	6 Argyll St	London	NW2	400
PG21	18 Dale Rd	Glasgow	G12	600

Client

clientNo	fName	IName	address	telNo
CR76	John	Kay	56 High St, London SW1 4EH	0171-774-5632
CR74	Mike	Ritchie	18 Tain St, PA1G 1YQ	01475-392178
CR62	Mary	Tregear	5 Tarbot Rd, Aberdeen AB9 3ST	01224-196720

Figure 1.4

The Lease, PropertyForRent, and Client files used by Contracts.

**Figure 1.5**

File-based processing.

Sales Files

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)

PrivateOwner (ownerNo, fName, IName, address, telNo)

Client (clientNo, fName, IName, address, telNo, prefType, maxRent)

Contracts Files

Lease (leaseNo, propertyNo, clientNo, rent, paymentMethod, deposit, paid, rentStart, rentFinish, duration)

PropertyForRent (propertyNo, street, city, postcode, rent)

Client (clientNo, fName, IName, address, telNo)

It can be seen quite clearly that there is a significant amount of duplication of data in these departments, and this is generally true of file-based systems. Before we discuss the limitations of this approach, it may be useful to understand the terminology used in file-based systems. A file is simply a collection of **records**, which contains **logically related**

data. For example, the `PropertyForRent` file in Figure 1.2 contains six records, one for each property. Each record contains a logically connected set of one or more **fields**, where each field represents some characteristic of the real-world object that is being modeled. In Figure 1.2, the fields of the `PropertyForRent` file represent characteristics of properties, such as address, property type, and number of rooms.

1.2.2 Limitations of the File-Based Approach

This brief description of traditional file-based systems should be sufficient to discuss the limitations of this approach. We list five problems in Table 1.1.

Separation and isolation of data

When data is isolated in separate files, it is more difficult to access data that should be available. For example, if we want to produce a list of all houses that match the requirements of clients, we first need to create a temporary file of those clients who have ‘house’ as the preferred type. We then search the `PropertyForRent` file for those properties where the property type is ‘house’ and the rent is less than the client’s maximum rent. With file systems, such processing is difficult. The application developer must synchronize the processing of two files to ensure the correct data is extracted. This difficulty is compounded if we require data from more than two files.

Duplication of data

Owing to the decentralized approach taken by each department, the file-based approach encouraged, if not necessitated, the uncontrolled duplication of data. For example, in Figure 1.5 we can clearly see that there is duplication of both property and client details in the Sales and Contracts Departments. Uncontrolled duplication of data is undesirable for several reasons, including:

- Duplication is wasteful. It costs time and money to enter the data more than once.
- It takes up additional storage space, again with associated costs. Often, the duplication of data can be avoided by sharing data files.
- Perhaps more importantly, duplication can lead to loss of data integrity; in other words, the data is no longer consistent. For example, consider the duplication of data between

Table 1.1 Limitations of file-based systems.

Separation and isolation of data
Duplication of data
Data dependence
Incompatible file formats
Fixed queries/proliferation of application programs

the Payroll and Personnel Departments described above. If a member of staff moves house and the change of address is communicated only to Personnel and not to Payroll, the person's payslip will be sent to the wrong address. A more serious problem occurs if an employee is promoted with an associated increase in salary. Again, the change is notified to Personnel but the change does not filter through to Payroll. Now, the employee is receiving the wrong salary. When this error is detected, it will take time and effort to resolve. Both these examples illustrate inconsistencies that may result from the duplication of data. As there is no automatic way for Personnel to update the data in the Payroll files, it is not difficult to foresee such inconsistencies arising. Even if Payroll is notified of the changes, it is possible that the data will be entered incorrectly.

Data dependence

As we have already mentioned, the physical structure and storage of the data files and records are defined in the application code. This means that changes to an existing structure are difficult to make. For example, increasing the size of the `PropertyForRent` address field from 40 to 41 characters sounds like a simple change, but it requires the creation of a one-off program (that is, a program that is run only once and can then be discarded) that converts the `PropertyForRent` file to the new format. This program has to:

- open the original `PropertyForRent` file for reading;
- open a temporary file with the new structure;
- read a record from the original file, convert the data to conform to the new structure, and write it to the temporary file. Repeat this step for all records in the original file;
- delete the original `PropertyForRent` file;
- rename the temporary file as `PropertyForRent`.

In addition, all programs that access the `PropertyForRent` file must be modified to conform to the new file structure. There might be many such programs that access the `PropertyForRent` file. Thus, the programmer needs to identify all the affected programs, modify them, and then retest them. Note that a program does not even have to use the address field to be affected: it has only to use the `PropertyForRent` file. Clearly, this could be very time-consuming and subject to error. This characteristic of file-based systems is known as **program–data dependence**.

Incompatible file formats

Because the structure of files is embedded in the application programs, the structures are dependent on the application programming language. For example, the structure of a file generated by a COBOL program may be different from the structure of a file generated by a 'C' program. The direct incompatibility of such files makes them difficult to process jointly.

For example, suppose that the Contracts Department wants to find the names and addresses of all owners whose property is currently rented out. Unfortunately, Contracts

does not hold the details of property owners; only the Sales Department holds these. However, Contracts has the property number (`propertyNo`), which can be used to find the corresponding property number in the Sales Department's `PropertyForRent` file. This file holds the owner number (`ownerNo`), which can be used to find the owner details in the `PrivateOwner` file. The Contracts Department programs in COBOL and the Sales Department programs in 'C'. Therefore, to match `propertyNo` fields in the two `PropertyForRent` files requires an application developer to write software to convert the files to some common format to facilitate processing. Again, this can be time-consuming and expensive.

Fixed queries/proliferation of application programs

From the end-user's point of view, file-based systems proved to be a great improvement over manual systems. Consequently, the requirement for new or modified queries grew. However, file-based systems are very dependent upon the application developer, who has to write any queries or reports that are required. As a result, two things happened. In some organizations, the type of query or report that could be produced was fixed. There was no facility for asking unplanned (that is, spur-of-the-moment or *ad hoc*) queries either about the data itself or about which types of data were available.

In other organizations, there was a proliferation of files and application programs. Eventually, this reached a point where the DP Department, with its current resources, could not handle all the work. This put tremendous pressure on the DP staff, resulting in programs that were inadequate or inefficient in meeting the demands of the users, documentation that was limited, and maintenance that was difficult. Often, certain types of functionality were omitted including:

- there was no provision for security or integrity;
- recovery, in the event of a hardware or software failure, was limited or non-existent;
- access to the files was restricted to one user at a time – there was no provision for shared access by staff in the same department.

In either case, the outcome was not acceptable. Another solution was required.

1.3 Database Approach

All the above limitations of the file-based approach can be attributed to two factors:

- (1) the definition of the data is embedded in the application programs, rather than being stored separately and independently;
- (2) there is no control over the access and manipulation of data beyond that imposed by the application programs.

To become more effective, a new approach was required. What emerged were the **database** and the **Database Management System (DBMS)**. In this section, we provide a more formal definition of these terms, and examine the components that we might expect in a DBMS environment.

The Database

1.3.1

Database A shared collection of logically related data, and a description of this data, designed to meet the information needs of an organization.

We now examine the definition of a database to understand the concept fully. The database is a single, possibly large repository of data that can be used simultaneously by many departments and users. Instead of disconnected files with redundant data, all data items are integrated with a minimum amount of duplication. The database is no longer owned by one department but is a shared corporate resource. The database holds not only the organization's operational data but also a description of this data. For this reason, a database is also defined as a *self-describing collection of integrated records*. The description of the data is known as the **system catalog** (or **data dictionary** or **metadata** – the 'data about data'). It is the self-describing nature of a database that provides **program–data independence**.

The approach taken with database systems, where the definition of data is separated from the application programs, is similar to the approach taken in modern software development, where an internal definition of an object and a separate external definition are provided. The users of an object see only the external definition and are unaware of how the object is defined and how it functions. One advantage of this approach, known as **data abstraction**, is that we can change the internal definition of an object without affecting the users of the object, provided the external definition remains the same. In the same way, the database approach separates the structure of the data from the application programs and stores it in the database. If new data structures are added or existing structures are modified then the application programs are unaffected, provided they do not directly depend upon what has been modified. For example, if we add a new field to a record or create a new file, existing applications are unaffected. However, if we remove a field from a file that an application program uses, then that application program is affected by this change and must be modified accordingly.

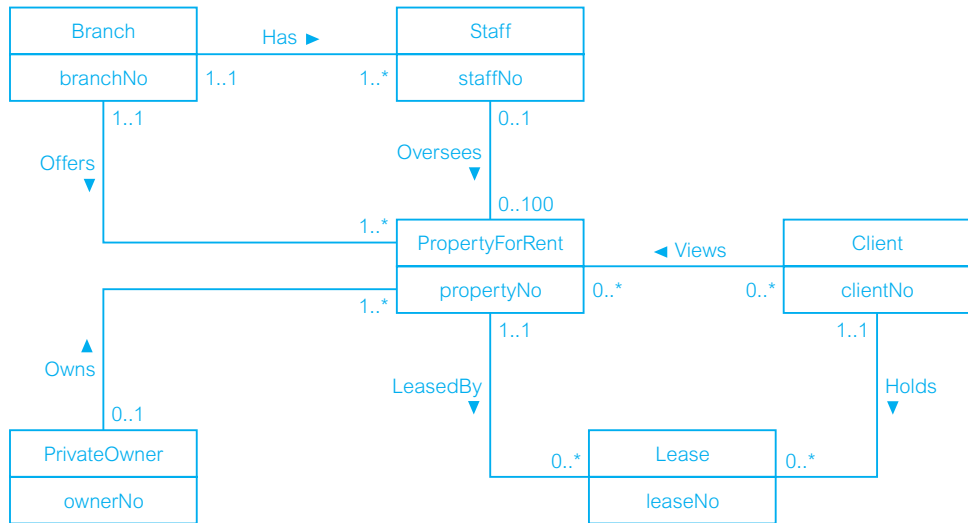
The final term in the definition of a database that we should explain is 'logically related'. When we analyze the information needs of an organization, we attempt to identify entities, attributes, and relationships. An **entity** is a distinct object (a person, place, thing, concept, or event) in the organization that is to be represented in the database. An **attribute** is a property that describes some aspect of the object that we wish to record, and a **relationship** is an association between entities. For example, Figure 1.6 shows an Entity–Relationship (ER) diagram for part of the *DreamHome* case study. It consists of:

- six entities (the rectangles): Branch, Staff, PropertyForRent, Client, PrivateOwner, and Lease;
- seven relationships (the names adjacent to the lines): *Has*, *Offers*, *Oversees*, *Views*, *Owns*, *LeasedBy*, and *Holds*;
- six attributes, one for each entity: branchNo, staffNo, propertyNo, clientNo, ownerNo, and leaseNo.

The database represents the entities, the attributes, and the logical relationships between the entities. In other words, the database holds data that is logically related. We discuss the Entity–Relationship model in detail in Chapters 11 and 12.

Figure 1.6

Example
Entity–Relationship
diagram.



1.3.2 The Database Management System (DBMS)

DBMS A software system that enables users to define, create, maintain, and control access to the database.

The DBMS is the software that interacts with the users' application programs and the database. Typically, a DBMS provides the following facilities:

- It allows users to define the database, usually through a **Data Definition Language** (DDL). The DDL allows users to specify the data types and structures and the constraints on the data to be stored in the database.
- It allows users to insert, update, delete, and retrieve data from the database, usually through a **Data Manipulation Language** (DML). Having a central repository for all data and data descriptions allows the DML to provide a general inquiry facility to this data, called a **query language**. The provision of a query language alleviates the problems with file-based systems where the user has to work with a fixed set of queries or there is a proliferation of programs, giving major software management problems. The most common query language is the **Structured Query Language** (SQL, pronounced 'S-Q-L', or sometimes 'See-Quel'), which is now both the formal and *de facto* standard language for relational DBMSs. To emphasize the importance of SQL, we devote Chapters 5 and 6, most of 28, and Appendix E to a comprehensive study of this language.
- It provides controlled access to the database. For example, it may provide:
 - a security system, which prevents unauthorized users accessing the database;
 - an integrity system, which maintains the consistency of stored data;
 - a concurrency control system, which allows shared access of the database;

- a recovery control system, which restores the database to a previous consistent state following a hardware or software failure;
- a user-accessible catalog, which contains descriptions of the data in the database.

(Database) Application Programs

1.3.3

Application program A computer program that interacts with the database by issuing an appropriate request (typically an SQL statement) to the DBMS.

Users interact with the database through a number of **application programs** that are used to create and maintain the database and to generate information. These programs can be conventional batch applications or, more typically nowadays, they will be online applications. The application programs may be written in some programming language or in some higher-level fourth-generation language.

The database approach is illustrated in Figure 1.7, based on the file approach of Figure 1.5. It shows the Sales and Contracts Departments using their application programs to access the database through the DBMS. Each set of departmental application programs handles data entry, data maintenance, and the generation of reports. However, compared with the file-based approach, the physical structure and storage of the data are now managed by the DBMS.

Views

With this functionality, the DBMS is an extremely powerful and useful tool. However, as the end-users are not too interested in how complex or easy a task is for the system, it could be argued that the DBMS has made things more complex because they now see

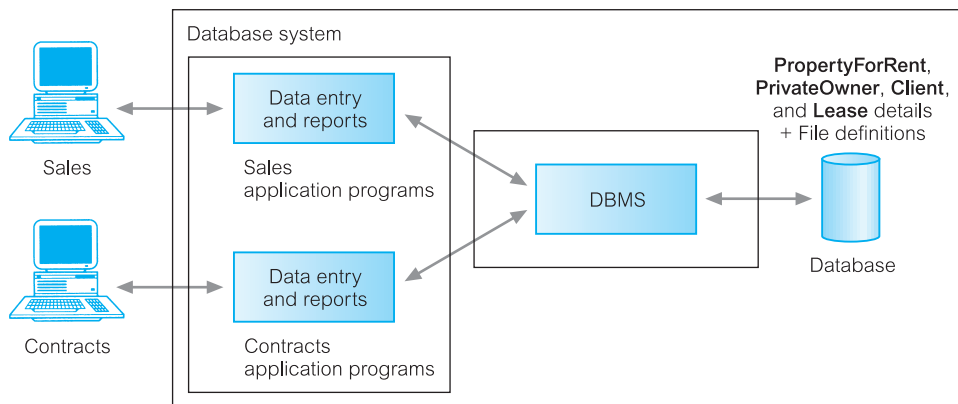


Figure 1.7
Database processing.

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo)

PrivateOwner (ownerNo, fName, lName, address, telNo)

Client (clientNo, fName, lName, address, telNo, prefType, maxRent)

Lease (leaseNo, propertyNo, clientNo, paymentMethod, deposit, paid, rentStart, rentFinish)

more data than they actually need or want. For example, the details that the Contracts Department wants to see for a rental property, as shown in Figure 1.5, have changed in the database approach, shown in Figure 1.7. Now the database also holds the property type, the number of rooms, and the owner details. In recognition of this problem, a DBMS provides another facility known as a **view mechanism**, which allows each user to have his or her own view of the database (a **view** is in essence some subset of the database). For example, we could set up a view that allows the Contracts Department to see only the data that they want to see for rental properties.

As well as reducing complexity by letting users see the data in the way they want to see it, views have several other benefits:

- *Views provide a level of security.* Views can be set up to exclude data that some users should not see. For example, we could create a view that allows a branch manager and the Payroll Department to see all staff data, including salary details, and we could create a second view that other staff would use that excludes salary details.
- *Views provide a mechanism to customize the appearance of the database.* For example, the Contracts Department may wish to call the monthly rent field (rent) by the more obvious name, Monthly Rent.
- *A view can present a consistent, unchanging picture of the structure of the database,* even if the underlying database is changed (for example, fields added or removed, relationships changed, files split, restructured, or renamed). If fields are added or removed from a file, and these fields are not required by the view, the view is not affected by this change. Thus, a view helps provide the program–data independence we mentioned in the previous section.

The above discussion is general and the actual level of functionality offered by a DBMS differs from product to product. For example, a DBMS for a personal computer may not support concurrent shared access, and it may provide only limited security, integrity, and recovery control. However, modern, large multi-user DBMS products offer all the above functions and much more. Modern systems are extremely complex pieces of software consisting of millions of lines of code, with documentation comprising many volumes. This is a result of having to provide software that handles requirements of a more general nature. Furthermore, the use of DBMSs nowadays requires a system that provides almost total reliability and 24/7 availability (24 hours a day, 7 days a week), even in the presence of hardware or software failure. The DBMS is continually evolving and expanding to cope with new user requirements. For example, some applications now require the storage of graphic images, video, sound, and so on. To reach this market, the DBMS must change. It is likely that new functionality will always be required, so that the functionality of the DBMS will never become static. We discuss the basic functions provided by a DBMS in later chapters.

1.3.4 Components of the DBMS Environment

We can identify five major components in the DBMS environment: hardware, software, data, procedures, and people, as illustrated in Figure 1.8.

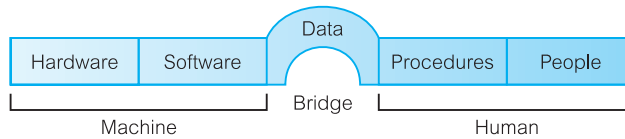


Figure 1.8
DBMS environment.

Hardware

The DBMS and the applications require hardware to run. The hardware can range from a single personal computer, to a single mainframe, to a network of computers. The particular hardware depends on the organization's requirements and the DBMS used. Some DBMSs run only on particular hardware or operating systems, while others run on a wide variety of hardware and operating systems. A DBMS requires a minimum amount of main memory and disk space to run, but this minimum configuration may not necessarily give acceptable performance. A simplified hardware configuration for *DreamHome* is illustrated in Figure 1.9. It consists of a network of minicomputers, with a central computer located in London running the **backend** of the DBMS, that is, the part of the DBMS that manages and controls access to the database. It also shows several computers at various

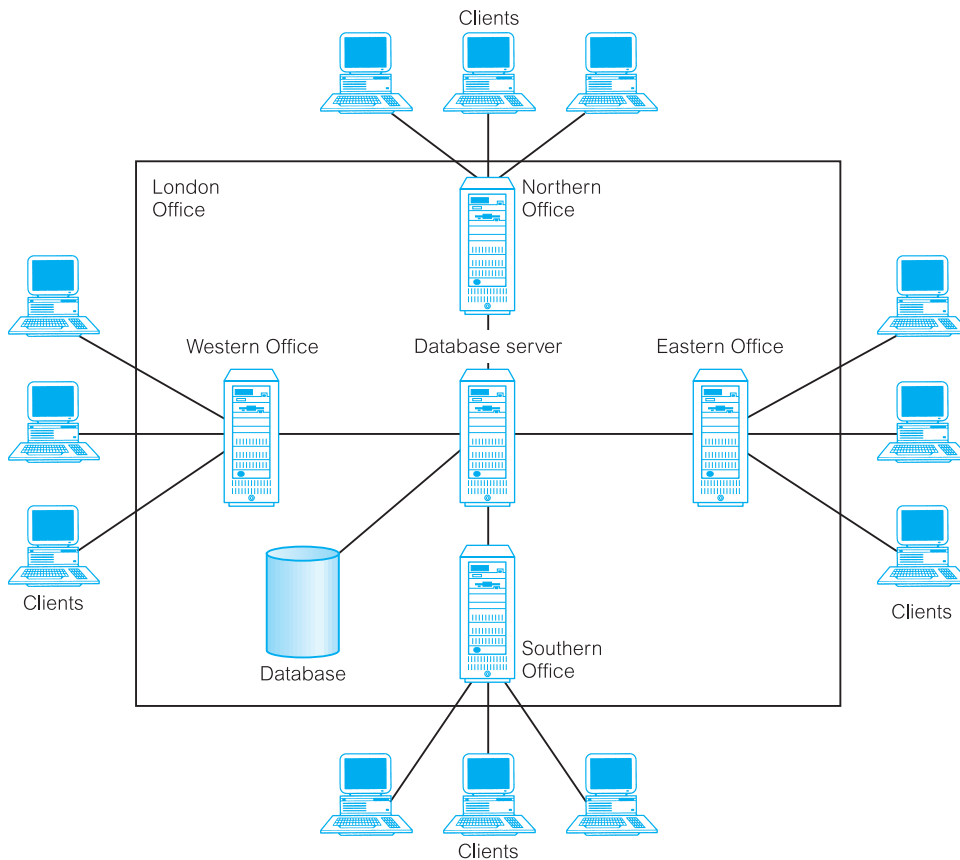


Figure 1.9
DreamHome
hardware
configuration.

locations running the **frontend** of the DBMS, that is, the part of the DBMS that interfaces with the user. This is called a **client–server** architecture: the backend is the server and the frontends are the clients. We discuss this type of architecture in Section 2.6.

Software

The software component comprises the DBMS software itself and the application programs, together with the operating system, including network software if the DBMS is being used over a network. Typically, application programs are written in a third-generation programming language (3GL), such as ‘C’, C++, Java, Visual Basic, COBOL, Fortran, Ada, or Pascal, or using a fourth-generation language (4GL), such as SQL, embedded in a third-generation language. The target DBMS may have its own fourth-generation tools that allow rapid development of applications through the provision of non-procedural query languages, reports generators, forms generators, graphics generators, and application generators. The use of fourth-generation tools can improve productivity significantly and produce programs that are easier to maintain. We discuss fourth-generation tools in Section 2.2.3.

Data

Perhaps the most important component of the DBMS environment, certainly from the end-users’ point of view, is the data. From Figure 1.8, we observe that the data acts as a bridge between the machine components and the human components. The database contains both the operational data and the metadata, the ‘data about data’. The structure of the database is called the **schema**. In Figure 1.7, the schema consists of four files, or **tables**, namely: PropertyForRent, PrivateOwner, Client, and Lease. The PropertyForRent table has eight fields, or **attributes**, namely: propertyNo, street, city, postcode, type (the property type), rooms (the number of rooms), rent (the monthly rent), and ownerNo. The ownerNo attribute models the relationship between PropertyForRent and PrivateOwner: that is, an owner *Owns* a property for rent, as depicted in the Entity–Relationship diagram of Figure 1.6. For example, in Figure 1.2 we observe that owner CO46, Joe Keogh, owns property PA14.

The data also incorporates the system catalog, which we discuss in detail in Section 2.4.

Procedures

Procedures refer to the instructions and rules that govern the design and use of the database. The users of the system and the staff that manage the database require documented procedures on how to use or run the system. These may consist of instructions on how to:

- log on to the DBMS;
- use a particular DBMS facility or application program;
- start and stop the DBMS;
- make backup copies of the database;
- handle hardware or software failures. This may include procedures on how to identify the failed component, how to fix the failed component (for example, telephone the appropriate hardware engineer) and, following the repair of the fault, how to recover the database;

- change the structure of a table, reorganize the database across multiple disks, improve performance, or archive data to secondary storage.

People

The final component is the people involved with the system. We discuss this component in Section 1.4.

Database Design: The Paradigm Shift

1.3.5

Until now, we have taken it for granted that there is a structure to the data in the database. For example, we have identified four tables in Figure 1.7: *PropertyForRent*, *PrivateOwner*, *Client*, and *Lease*. But how did we get this structure? The answer is quite simple: the structure of the database is determined during **database design**. However, carrying out database design can be extremely complex. To produce a system that will satisfy the organization's information needs requires a different approach from that of file-based systems, where the work was driven by the application needs of individual departments. For the database approach to succeed, the organization now has to think of the data first and the application second. This change in approach is sometimes referred to as a *paradigm shift*. For the system to be acceptable to the end-users, the database design activity is crucial. A poorly designed database will generate errors that may lead to bad decisions being made, which may have serious repercussions for the organization. On the other hand, a well-designed database produces a system that provides the correct information for the decision-making process to succeed in an efficient way.

The objective of this book is to help effect this paradigm shift. We devote several chapters to the presentation of a complete methodology for database design (see Chapters 15–18). It is presented as a series of simple-to-follow steps, with guidelines provided throughout. For example, in the Entity–Relationship diagram of Figure 1.6, we have identified six entities, seven relationships, and six attributes. We provide guidelines to help identify the entities, attributes, and relationships that have to be represented in the database.

Unfortunately, database design methodologies are not very popular. Many organizations and individual designers rely very little on methodologies for conducting the design of databases, and this is commonly considered a major cause of failure in the development of database systems. Owing to the lack of structured approaches to database design, the time or resources required for a database project are typically underestimated, the databases developed are inadequate or inefficient in meeting the demands of applications, documentation is limited, and maintenance is difficult.

Roles in the Database Environment

1.4

In this section, we examine what we listed in the previous section as the fifth component of the DBMS environment: the **people**. We can identify four distinct types of people that participate in the DBMS environment: data and database administrators, database designers, application developers, and the end-users.

1.4.1 Data and Database Administrators

The database and the DBMS are corporate resources that must be managed like any other resource. Data and database administration are the roles generally associated with the management and control of a DBMS and its data. The **Data Administrator (DA)** is responsible for the management of the data resource including database planning, development and maintenance of standards, policies and procedures, and conceptual/logical database design. The DA consults with and advises senior managers, ensuring that the direction of database development will ultimately support corporate objectives.

The **Database Administrator (DBA)** is responsible for the physical realization of the database, including physical database design and implementation, security and integrity control, maintenance of the operational system, and ensuring satisfactory performance of the applications for users. The role of the DBA is more technically oriented than the role of the DA, requiring detailed knowledge of the target DBMS and the system environment. In some organizations there is no distinction between these two roles; in others, the importance of the corporate resources is reflected in the allocation of teams of staff dedicated to each of these roles. We discuss data and database administration in more detail in Section 9.15.

1.4.2 Database Designers

In large database design projects, we can distinguish between two types of designer: logical database designers and physical database designers. The **logical database designer** is concerned with identifying the data (that is, the entities and attributes), the relationships between the data, and the constraints on the data that is to be stored in the database. The logical database designer must have a thorough and complete understanding of the organization's data and any constraints on this data (the constraints are sometimes called **business rules**). These constraints describe the main characteristics of the data as viewed by the organization. Examples of constraints for *DreamHome* are:

- a member of staff cannot manage more than 100 properties for rent or sale at the same time;
- a member of staff cannot handle the sale or rent of his or her own property;
- a solicitor cannot act for both the buyer and seller of a property.

To be effective, the logical database designer must involve all prospective database users in the development of the data model, and this involvement should begin as early in the process as possible. In this book, we split the work of the logical database designer into two stages:

- conceptual database design, which is independent of implementation details such as the target DBMS, application programs, programming languages, or any other physical considerations;
- logical database design, which targets a specific data model, such as relational, network, hierarchical, or object-oriented.

The **physical database designer** decides how the logical database design is to be physically realized. This involves:

- mapping the logical database design into a set of tables and integrity constraints;
- selecting specific storage structures and access methods for the data to achieve good performance;
- designing any security measures required on the data.

Many parts of physical database design are highly dependent on the target DBMS, and there may be more than one way of implementing a mechanism. Consequently, the physical database designer must be fully aware of the functionality of the target DBMS and must understand the advantages and disadvantages of each alternative for a particular implementation. The physical database designer must be capable of selecting a suitable storage strategy that takes account of usage. Whereas conceptual and logical database design are concerned with the *what*, physical database design is concerned with the *how*. It requires different skills, which are often found in different people. We present a methodology for conceptual database design in Chapter 15, for logical database design in Chapter 16, and for physical database design in Chapters 17 and 18.

Application Developers

1.4.3

Once the database has been implemented, the application programs that provide the required functionality for the end-users must be implemented. This is the responsibility of the **application developers**. Typically, the application developers work from a specification produced by systems analysts. Each program contains statements that request the DBMS to perform some operation on the database. This includes retrieving data, inserting, updating, and deleting data. The programs may be written in a third-generation programming language or a fourth-generation language, as discussed in the previous section.

End-Users

1.4.4

The end-users are the ‘clients’ for the database, which has been designed and implemented, and is being maintained to serve their information needs. End-users can be classified according to the way they use the system:

- **Naïve users** are typically unaware of the DBMS. They access the database through specially written application programs that attempt to make the operations as simple as possible. They invoke database operations by entering simple commands or choosing options from a menu. This means that they do not need to know anything about the database or the DBMS. For example, the checkout assistant at the local supermarket uses a bar code reader to find out the price of the item. However, there is an application program present that reads the bar code, looks up the price of the item in the database, reduces the database field containing the number of such items in stock, and displays the price on the till.

- **Sophisticated users.** At the other end of the spectrum, the sophisticated end-user is familiar with the structure of the database and the facilities offered by the DBMS. Sophisticated end-users may use a high-level query language such as SQL to perform the required operations. Some sophisticated end-users may even write application programs for their own use.

1.5

History of Database Management Systems

We have already seen that the predecessor to the DBMS was the file-based system. However, there was never a time when the database approach began and the file-based system ceased. In fact, the file-based system still exists in specific areas. It has been suggested that the DBMS has its roots in the 1960s Apollo moon-landing project, which was initiated in response to President Kennedy's objective of landing a man on the moon by the end of that decade. At that time there was no system available that would be able to handle and manage the vast amounts of information that the project would generate.

As a result, North American Aviation (NAA, now Rockwell International), the prime contractor for the project, developed software known as **GUAM** (Generalized Update Access Method). GUAM was based on the concept that smaller components come together as parts of larger components, and so on, until the final product is assembled. This structure, which conforms to an upside-down tree, is also known as a **hierarchical structure**. In the mid-1960s, IBM joined NAA to develop GUAM into what is now known as **IMS** (Information Management System). The reason why IBM restricted IMS to the management of hierarchies of records was to allow the use of serial storage devices, most notably magnetic tape, which was a market requirement at that time. This restriction was subsequently dropped. Although one of the earliest commercial DBMSs, IMS is still the main hierarchical DBMS used by most large mainframe installations.

In the mid-1960s, another significant development was the emergence of **IDS** (Integrated Data Store) from General Electric. This work was headed by one of the early pioneers of database systems, Charles Bachmann. This development led to a new type of database system known as the **network DBMS**, which had a profound effect on the information systems of that generation. The network database was developed partly to address the need to represent more complex data relationships than could be modeled with hierarchical structures, and partly to impose a database standard. To help establish such standards, the Conference on Data Systems Languages (**CODASYL**), comprising representatives of the US government and the world of business and commerce, formed a List Processing Task Force in 1965, subsequently renamed the **Data Base Task Group** (DBTG) in 1967. The terms of reference for the DBTG were to define standard specifications for an environment that would allow database creation and data manipulation. A draft report was issued in 1969 and the first definitive report in 1971. The DBTG proposal identified three components:

- the network **schema** – the logical organization of the entire database as seen by the DBA – which includes a definition of the database name, the type of each record, and the components of each record type;
- the **subschemas** – the part of the database as seen by the user or application program;
- a data management language to define the data characteristics and the data structure, and to manipulate the data.

For standardization, the DBTG specified three distinct languages:

- a schema **Data Definition Language** (DDL), which enables the DBA to define the schema;
- a subschema **DDL**, which allows the application programs to define the parts of the database they require;
- a **Data Manipulation Language** (DML), to manipulate the data.

Although the report was not formally adopted by the American National Standards Institute (ANSI), a number of systems were subsequently developed following the DBTG proposal. These systems are now known as CODASYL or DBTG systems. The CODASYL and hierarchical approaches represented the **first-generation** of DBMSs. We look more closely at these systems on the Web site for this book (see Preface for the URL). However, these two models have some fundamental disadvantages:

- complex programs have to be written to answer even simple queries based on navigational record-oriented access;
- there is minimal data independence;
- there is no widely accepted theoretical foundation.

In 1970 E. F. Codd of the IBM Research Laboratory produced his highly influential paper on the relational data model. This paper was very timely and addressed the disadvantages of the former approaches. Many experimental relational DBMSs were implemented thereafter, with the first commercial products appearing in the late 1970s and early 1980s. Of particular note is the System R project at IBM's San José Research Laboratory in California, which was developed during the late 1970s (Astrahan *et al.*, 1976). This project was designed to prove the practicality of the relational model by providing an implementation of its data structures and operations, and led to two major developments:

- the development of a structured query language called SQL, which has since become the standard language for relational DBMSs;
- the production of various commercial relational DBMS products during the 1980s, for example DB2 and SQL/DS from IBM and Oracle from Oracle Corporation.

Now there are several hundred relational DBMSs for both mainframe and PC environments, though many are stretching the definition of the relational model. Other examples of multi-user relational DBMSs are Advantage Ingres Enterprise Relational Database from Computer Associates, and Informix from IBM. Examples of PC-based relational DBMSs are Office Access and Visual FoxPro from Microsoft, InterBase and JDataStore from Borland, and R:Base from R:Base Technologies. Relational DBMSs are referred to as **second-generation** DBMSs. We discuss the relational data model in Chapter 3.

The relational model is not without its failings, and in particular its limited modeling capabilities. There has been much research since then attempting to address this problem. In 1976, Chen presented the Entity–Relationship model, which is now a widely accepted technique for database design and the basis for the methodology presented in Chapters 15 and 16 of this book. In 1979, Codd himself attempted to address some of the failings in his original work with an extended version of the relational model called RM/T (1979) and subsequently RM/V2 (1990). The attempts to provide a data model that represents the ‘real world’ more closely have been loosely classified as **semantic data modeling**.

In response to the increasing complexity of database applications, two ‘new’ systems have emerged: the **Object-Oriented DBMS** (OODBMS) and the **Object-Relational DBMS** (ORDBMS). However, unlike previous models, the actual composition of these models is not clear. This evolution represents **third-generation** DBMSs, which we discuss in Chapters 25–28.

1.6

Advantages and Disadvantages of DBMSs

The database management system has promising potential advantages. Unfortunately, there are also disadvantages. In this section, we examine these advantages and disadvantages.

Advantages

The advantages of database management systems are listed in Table 1.2.

Control of data redundancy

As we discussed in Section 1.2, traditional file-based systems waste space by storing the same information in more than one file. For example, in Figure 1.5, we stored similar data for properties for rent and clients in both the Sales and Contracts Departments. In contrast, the database approach attempts to eliminate the redundancy by integrating the files so that multiple copies of the same data are not stored. However, the database approach does not eliminate redundancy entirely, but controls the amount of redundancy inherent in the database. Sometimes, it is necessary to duplicate key data items to model relationships. At other times, it is desirable to duplicate some data items to improve performance. The reasons for controlled duplication will become clearer as you read the next few chapters.

Data consistency

By eliminating or controlling redundancy, we reduce the risk of inconsistencies occurring. If a data item is stored only once in the database, any update to its value has to be performed only once and the new value is available immediately to all users. If a data item is stored more than once and the system is aware of this, the system can ensure that all copies

Table 1.2 Advantages of DBMSs.

Control of data redundancy	Economy of scale
Data consistency	Balance of conflicting requirements
More information from the same amount of data	Improved data accessibility and responsiveness
Sharing of data	Increased productivity
Improved data integrity	Improved maintenance through data independence
Improved security	Increased concurrency
Enforcement of standards	Improved backup and recovery services

of the item are kept consistent. Unfortunately, many of today's DBMSs do not automatically ensure this type of consistency.

More information from the same amount of data

With the integration of the operational data, it may be possible for the organization to derive additional information from the same data. For example, in the file-based system illustrated in Figure 1.5, the Contracts Department does not know who owns a leased property. Similarly, the Sales Department has no knowledge of lease details. When we integrate these files, the Contracts Department has access to owner details and the Sales Department has access to lease details. We may now be able to derive more information from the same amount of data.

Sharing of data

Typically, files are owned by the people or departments that use them. On the other hand, the database belongs to the entire organization and can be shared by all authorized users. In this way, more users share more of the data. Furthermore, new applications can build on the existing data in the database and add only data that is not currently stored, rather than having to define all data requirements again. The new applications can also rely on the functions provided by the DBMS, such as data definition and manipulation, and concurrency and recovery control, rather than having to provide these functions themselves.

Improved data integrity

Database integrity refers to the validity and consistency of stored data. Integrity is usually expressed in terms of **constraints**, which are consistency rules that the database is not permitted to violate. Constraints may apply to data items within a single record or they may apply to relationships between records. For example, an integrity constraint could state that a member of staff's salary cannot be greater than £40,000 or that the branch number contained in a staff record, representing the branch where the member of staff works, must correspond to an existing branch office. Again, integration allows the DBA to define, and the DBMS to enforce, integrity constraints.

Improved security

Database security is the protection of the database from unauthorized users. Without suitable security measures, integration makes the data more vulnerable than file-based systems. However, integration allows the DBA to define, and the DBMS to enforce, database security. This may take the form of user names and passwords to identify people authorized to use the database. The access that an authorized user is allowed on the data may be restricted by the operation type (retrieval, insert, update, delete). For example, the DBA has access to all the data in the database; a branch manager may have access to all data that relates to his or her branch office; and a sales assistant may have access to all data relating to properties but no access to sensitive data such as staff salary details.

Enforcement of standards

Again, integration allows the DBA to define and enforce the necessary standards. These may include departmental, organizational, national, or international standards for such

things as data formats to facilitate exchange of data between systems, naming conventions, documentation standards, update procedures, and access rules.

Economy of scale

Combining all the organization's operational data into one database, and creating a set of applications that work on this one source of data, can result in cost savings. In this case, the budget that would normally be allocated to each department for the development and maintenance of its file-based system can be combined, possibly resulting in a lower total cost, leading to an economy of scale. The combined budget can be used to buy a system configuration that is more suited to the organization's needs. This may consist of one large, powerful computer or a network of smaller computers.

Balance of conflicting requirements

Each user or department has needs that may be in conflict with the needs of other users. Since the database is under the control of the DBA, the DBA can make decisions about the design and operational use of the database that provide the best use of resources for the organization as a whole. These decisions will provide optimal performance for important applications, possibly at the expense of less critical ones.

Improved data accessibility and responsiveness

Again, as a result of integration, data that crosses departmental boundaries is directly accessible to the end-users. This provides a system with potentially much more functionality that can, for example, be used to provide better services to the end-user or the organization's clients. Many DBMSs provide query languages or report writers that allow users to ask *ad hoc* questions and to obtain the required information almost immediately at their terminal, without requiring a programmer to write some software to extract this information from the database. For example, a branch manager could list all flats with a monthly rent greater than £400 by entering the following SQL command at a terminal:

```
SELECT*  
FROM PropertyForRent  
WHERE type = 'Flat' AND rent > 400;
```

Increased productivity

As mentioned previously, the DBMS provides many of the standard functions that the programmer would normally have to write in a file-based application. At a basic level, the DBMS provides all the low-level file-handling routines that are typical in application programs. The provision of these functions allows the programmer to concentrate on the specific functionality required by the users without having to worry about low-level implementation details. Many DBMSs also provide a fourth-generation environment consisting of tools to simplify the development of database applications. This results in increased programmer productivity and reduced development time (with associated cost savings).

Improved maintenance through data independence

In file-based systems, the descriptions of the data and the logic for accessing the data are built into each application program, making the programs dependent on the data. A

change to the structure of the data, for example making an address 41 characters instead of 40 characters, or a change to the way the data is stored on disk, can require substantial alterations to the programs that are affected by the change. In contrast, a DBMS separates the data descriptions from the applications, thereby making applications immune to changes in the data descriptions. This is known as **data independence** and is discussed further in Section 2.1.5. The provision of data independence simplifies database application maintenance.

Increased concurrency

In some file-based systems, if two or more users are allowed to access the same file simultaneously, it is possible that the accesses will interfere with each other, resulting in loss of information or even loss of integrity. Many DBMSs manage concurrent database access and ensure such problems cannot occur. We discuss concurrency control in Chapter 20.

Improved backup and recovery services

Many file-based systems place the responsibility on the user to provide measures to protect the data from failures to the computer system or application program. This may involve taking a nightly backup of the data. In the event of a failure during the next day, the backup is restored and the work that has taken place since this backup is lost and has to be re-entered. In contrast, modern DBMSs provide facilities to minimize the amount of processing that is lost following a failure. We discuss database recovery in Section 20.3.

Disadvantages

The disadvantages of the database approach are summarized in Table 1.3.

Complexity

The provision of the functionality we expect of a good DBMS makes the DBMS an extremely complex piece of software. Database designers and developers, the data and database administrators, and end-users must understand this functionality to take full advantage of it. Failure to understand the system can lead to bad design decisions, which can have serious consequences for an organization.

Table 1.3 Disadvantages of DBMSs.

Complexity
Size
Cost of DBMSs
Additional hardware costs
Cost of conversion
Performance
Higher impact of a failure

Size

The complexity and breadth of functionality makes the DBMS an extremely large piece of software, occupying many megabytes of disk space and requiring substantial amounts of memory to run efficiently.

Cost of DBMSs

The cost of DBMSs varies significantly, depending on the environment and functionality provided. For example, a single-user DBMS for a personal computer may only cost US\$100. However, a large mainframe multi-user DBMS servicing hundreds of users can be extremely expensive, perhaps US\$100,000 or even US\$1,000,000. There is also the recurrent annual maintenance cost, which is typically a percentage of the list price.

Additional hardware costs

The disk storage requirements for the DBMS and the database may necessitate the purchase of additional storage space. Furthermore, to achieve the required performance, it may be necessary to purchase a larger machine, perhaps even a machine dedicated to running the DBMS. The procurement of additional hardware results in further expenditure.

Cost of conversion

In some situations, the cost of the DBMS and extra hardware may be insignificant compared with the cost of converting existing applications to run on the new DBMS and hardware. This cost also includes the cost of training staff to use these new systems, and possibly the employment of specialist staff to help with the conversion and running of the system. This cost is one of the main reasons why some organizations feel tied to their current systems and cannot switch to more modern database technology. The term **legacy system** is sometimes used to refer to an older, and usually inferior, system.

Performance

Typically, a file-based system is written for a specific application, such as invoicing. As a result, performance is generally very good. However, the DBMS is written to be more general, to cater for many applications rather than just one. The effect is that some applications may not run as fast as they used to.

Higher impact of a failure

The centralization of resources increases the vulnerability of the system. Since all users and applications rely on the availability of the DBMS, the failure of certain components can bring operations to a halt.

Chapter Summary

- The **Database Management System (DBMS)** is now the underlying framework of the information system and has fundamentally changed the way that many organizations operate. The database system remains a very active research area and many significant problems have still to be satisfactorily resolved.
- The predecessor to the DBMS was the **file-based system**, which is a collection of application programs that perform services for the end-users, usually the production of reports. Each program defines and manages its own data. Although the file-based system was a great improvement on the manual filing system, it still has significant problems, mainly the amount of data redundancy present and program–data dependence.
- The database approach emerged to resolve the problems with the file-based approach. A **database** is a shared collection of logically related data, and a description of this data, designed to meet the information needs of an organization. A **DBMS** is a software system that enables users to define, create, maintain, and control access to the database. An **application program** is a computer program that interacts with the database by issuing an appropriate request (typically an SQL statement) to the DBMS. The more inclusive term **database system** is used to define a collection of application programs that interact with the database along with the DBMS and database itself.
- All access to the database is through the DBMS. The DBMS provides a **Data Definition Language (DDL)**, which allows users to define the database, and a **Data Manipulation Language (DML)**, which allows users to insert, update, delete, and retrieve data from the database.
- The DBMS provides controlled access to the database. It provides security, integrity, concurrency and recovery control, and a user-accessible catalog. It also provides a view mechanism to simplify the data that users have to deal with.
- The DBMS environment consists of hardware (the computer), software (the DBMS, operating system, and applications programs), data, procedures, and people. The people include data and database administrators, database designers, application developers, and end-users.
- The roots of the DBMS lie in file-based systems. The hierarchical and CODASYL systems represent the first-generation of DBMSs. The **hierarchical model** is typified by IMS (Information Management System) and the **network** or **CODASYL model** by IDS (Integrated Data Store), both developed in the mid-1960s. The **relational model**, proposed by E. F. Codd in 1970, represents the second-generation of DBMSs. It has had a fundamental effect on the DBMS community and there are now over one hundred relational DBMSs. The third-generation of DBMSs are represented by the **Object-Relational DBMS** and the **Object-Oriented DBMS**.
- Some advantages of the database approach include control of data redundancy, data consistency, sharing of data, and improved security and integrity. Some disadvantages include complexity, cost, reduced performance, and higher impact of a failure.

Review Questions

- 1.1 List four examples of database systems other than those listed in Section 1.1.
- 1.2 Discuss each of the following terms:
 - (a) data
 - (b) database
 - (c) database management system
 - (d) database application program
 - (e) data independence
 - (f) security
 - (g) integrity
 - (h) views.
- 1.3 Describe the approach taken to the handling of data in the early file-based systems. Discuss the disadvantages of this approach.
- 1.4 Describe the main characteristics of the database approach and contrast it with the file-based approach.
- 1.5 Describe the five components of the DBMS environment and discuss how they relate to each other.
- 1.6 Discuss the roles of the following personnel in the database environment:
 - (a) data administrator
 - (b) database administrator
 - (c) logical database designer
 - (d) physical database designer
 - (e) application developer
 - (f) end-users.
- 1.7 Discuss the advantages and disadvantages of DBMSs.

Exercises

- 1.8 Interview some users of database systems. Which DBMS features do they find most useful and why? Which DBMS facilities do they find least useful and why? What do these users perceive to be the advantages and disadvantages of the DBMS?
 - 1.9 Write a small program (using pseudocode if necessary) that allows entry and display of client details including a client number, name, address, telephone number, preferred number of rooms, and maximum rent. The details should be stored in a file. Enter a few records and display the details. Now repeat this process but rather than writing a special program, use any DBMS that you have access to. What can you conclude from these two approaches?
 - 1.10 Study the *DreamHome* case study presented in Section 10.4 and Appendix A. In what ways would a DBMS help this organization? What data can you identify that needs to be represented in the database? What relationships exist between the data items? What queries do you think are required?
 - 1.11 Study the *Wellmeadows Hospital* case study presented in Appendix B.3. In what ways would a DBMS help this organization? What data can you identify that needs to be represented in the database? What relationships exist between the data items?
-